STAGE 00 / TOOL FOUNDATIONS

A reproducible first run and a method for getting unstuck

Use mise, Cargo, and a nine-step investigation to distinguish shell, compiler, and program failures.

Read with a terminal open. Predict before revealing. Verified source versions: version ledger.

▼ **Prerequisites and learning objectives**

- Verify the active toolchain without changing a global default
- Read commands as a chain of boundaries
- Record an evidence-based investigation

Key terms: future · mise

> Keep the problem-solving playbook open. Record observations before explanations, and save the experiment that changed your mind.

Reconstruct this project's runnable checkpoints. Each milestone states its own scope; learner scaffolds are separate assignments.

Purpose and the first useful model

Your first deliverable is a reproducible calculator run, accompanied by an investigation that another learner can repeat. A terminal command is a request to several programs. The shell first divides text into arguments. mise chooses tools from project configuration. Cargo selects a package, builds it if necessary, and starts a binary. The binary parses the arguments that survive those earlier steps. A failure at one boundary can resemble a failure somewhere else. Learning the boundaries gives you a way to solve unfamiliar problems without memorizing every diagnostic.

This course uses mise. The course mise.toml pins Rust 1.99.0, the stable release announced on October 1, 2026. Begin in the course root, the folder containing index.html, mise.toml, scripts and projects; its folder name may differ from rust-course. The launch guide lists the mise, Python 3.11+, Git and native development-tool prerequisites. Open and inspect mise.toml before trusting it: it selects the pinned Rust version, a shared local build directory, and local Python tasks. Then run the setup and version commands below. rustup may be the underlying

mechanism used to manage the Rust toolchain; it is not a reason to change your global default. The matching rust-toolchain.toml records the same version for Rust-aware tools. If these disagree, stop to inspect configuration instead of updating dependencies until the error disappears.

```
Example · use the stated working directory


 pwd
ls
mise trust
mise install
mise exec -- rustc --version
mise exec -- cargo --version
cd projects
mise exec -- cargo run -p workshop-calculator -- 12m + 3m
```

The final line should print 15m. Cargo may also print compilation messages to standard error. The double dash ends Cargo's arguments; the following three arguments belong to the calculator. A successful command is evidence that this package builds and this case works. It does not establish that every project, every boundary case, or an arbitrary future Rust version works. Keep that distinction visible in your notes from the beginning.

Prediction before running

Predict what happens if the command omits Cargo's double dash: mise exec -- cargo run -p workshop-calculator 12m + 3m. Does the calculator receive three operands, report an unknown operator, or fail to start? Write both your answer and which program you think produces the error. Then run the command. Expected result: Cargo rejects the unexpected arguments before the calculator starts. The operand text never reaches our program. This is why a calculator patch cannot fix this particular reproduction.

Now predict the result of the correctly separated command with 12m + 3s. Expected result: the binary starts, prints a units error to standard error, and exits unsuccessfully. Cargo did its job. Our domain deliberately rejects adding a length and a time. A nonzero exit is sometimes the correct implementation of the requested behavior. The first question is always what behavior the contract requires.

The problem-solving playbook: a worked investigation

Keep the playbook reference beside your terminal. The following investigation models its complete loop; later lessons remove some of the prompts so you choose the next experiment yourself.

Step 1 — state expected and actual behavior precisely. Expected: three calculator arguments produce 15m. Actual for the missing-separator command: Cargo reports an unexpected argument; no quantity is printed. Avoid writing only “Rust is broken.” That phrase hides both the action and the observable difference.

Step 2 — reproduce with exact context. Record the working directory rust-course/projects, the full command, the output, and the output of mise exec -- rustc --version. The intended version begins with rustc 1.99.0. If yours differs, that is an observation, not permission to assume it caused the argument error. Record the active Cargo version too. Reproduction is a controlled description, not a collection of screenshots with missing context.

Step 3 — locate the boundary. The message names Cargo's command syntax, so investigate Cargo-to-binary argument forwarding. Step 4 — read evidence rather than only the last line. Cargo help describes the separator before executable arguments. The reference binary expects exactly three arguments after its own executable name. These facts narrow the search before any source edit.

Step 5 — separate competing hypotheses. Hypothesis A: the toolchain cannot compile the project. Hypothesis B: arguments are being interpreted by Cargo. A remains plausible until a normal run builds; B predicts that inserting one separator will change which program responds. We have not yet proved either hypothesis merely by naming it.

Step 6 — make one distinguishing experiment. Add only the double dash and keep package, directory, operands, and tool version constant. Expected experimental result: the normal command builds or reuses its build and prints 15m. This disproves A as the explanation for this reproduction, while supporting B. It does not prove that the installation can never fail for another package.

Step 7 — apply the smallest sound fix. Correct the documented invocation; do not rewrite argument parsing to accommodate an argument list that never arrived. Step 8 — regress and state limits. Run the good addition, an incompatible-unit case, and the package tests. These checks cover routing into the binary and some calculator contracts, not arbitrary shell quoting or operating-system behavior.

Step 9 — capture the result. Write a short note: “Missing Cargo separator; observed Cargo argument rejection; changed only --; calculator then produced 15m; recorded Rust 1.99.0; package tests still need to be run or their actual result attached.” Distinguish an expected example from a command you actually executed. This habit makes your later debugging notes usable evidence.

Guided checkpoint

Read the small entry point below. Identify where argument count becomes a usage error, where calculation errors become exit code 1, and where success becomes exit code 0. Trace 12m + 3m through those branches on paper before running the test command. The library is responsible for quantities; the entry point is responsible for interaction with a process. Keeping those responsibilities separate allows unit tests without launching a terminal.

projects/reference/p01-calculator/src/main.rs

```
 use std::{
    io::{self, Write},
    process::ExitCode,
};
fn run() -> io::Result<ExitCode> {
    let args = std::env::args_os()
```

```
    .skip(1)
    .map(|value| value.into_string())
    .collect::<Result<Vec<_>, _>>();
let mut stdout = io::stdout().lock();
let mut stderr = io::stderr().lock();
let Ok(args) = args else {
    writeln!(stderr, "calculator: arguments must be valid UTF-8 text")?;
    return Ok(ExitCode::from(2));
};
let [left, op, right] = args.as_slice() else {
    writeln!(
        stderr,
        "usage: workshop-calculator <integer[m|s]> <+|-|*|/> <integer[m|s]>"
    )?;
    writeln!(
        stderr,
        "       workshop-calculator convert <finite-number><mm|cm|m|km> <mm|cm|m|km>"
    )?;
    return Ok(ExitCode::from(2));
};
if left == "convert" {
    return match workshop_calculator::conversion::convert(op, right) {
        Ok(length) => {
            writeln!(
                stdout,
                "approximately {}{}",
                length.value(),
                length.unit().symbol()
            )?;
            stdout.flush()?;
            Ok(ExitCode::SUCCESS)
        }
        Err(error) => {
            writeln!(stderr, "conversion: {error}")?;
            Ok(ExitCode::FAILURE)
        }
    };
}
match workshop_calculator::calculate(left, op, right) {
    Ok(value) => {
        writeln!(stdout, "{value}")?;
        stdout.flush()?;
        Ok(ExitCode::SUCCESS)
    }
    Err(error) => {
        writeln!(stderr, "calculator: {error}")?;
        Ok(ExitCode::FAILURE)
    }
```

```
    }
}
fn main() -> ExitCode {
    match run() {
        Ok(code) => code,
        Err(error) => {
            // Diagnostic output is best effort; its failure cannot restore stdout.
            let _ = writeln!(io::stderr().lock(), "calculator output: {error}");
            ExitCode::FAILURE
        }
    }
}
```

```
Example · use the stated working directory


 mise exec -- cargo test -p workshop-calculator
mise exec -- cargo run -p workshop-calculator -- 12m + 3s
```

When you get stuck: independent work and tiered help

Make a three-row evidence table for a good invocation, missing operand, and division by zero. Record producing component, expected exit status, actual status, and output stream. Quote multiplication as '*' so the shell does not expand it into filenames. A review request should include the exact command and a single disputed conclusion: “I think the shell expanded the star before Cargo; what experiment would distinguish that from a parser error?”

▼ Hint 1: choose the next observation

count argument boundaries before reading calculator internals.

▼ Hint 2: narrow the experiment

compare the CLI integration test with the command that failed.

▼ **Worked solution**

use mise exec -- cargo run -p workshop-calculator -- 4 '*' 3. The quotes are interpreted by the shell and the binary receives the one-character operator. A request to AI should ask for competing explanations and a distinguishing experiment before requesting a patch; verify the proposed command against help and observed output.

Review and mastery evidence

The recommended workflow records the responsible program, the changed factor, and the reason the corrected invocation works. Reading Cargo help is the right abstraction for argument forwarding; an argument-parsing dependency would solve a different problem inside the binary. An alternative is to run the already-built executable directly when isolating Cargo from process behavior, while remembering to rebuild after source changes. Choose that experiment when the build/launch boundary is uncertain.

Primary references

- Rust 1.99.0 release
- Cargo run reference
- mise Rust tools